

EventZen

Product Requirements Document

Version: 4.0

Date: March 19, 2026

Status: Updated to reflect implemented platform changes

Prepared by: EventZen Engineering and Product Team

1. Executive Summary

EventZen has evolved from a two-surface event platform into a three-portal product with distinct customer, vendor, and admin experiences. Since the previous PRD revision, the codebase has added substantial backend API coverage across authentication, event management, ticketing, finance, venue/vendor operations, and notifications. The frontend has also expanded to support dedicated portal dashboards, self-service account workflows, finance operations, check-in, reporting, and vendor onboarding.

This PRD captures the current implemented product shape based on the repository state as of March 19, 2026. It is intended to serve as the updated functional and technical reference for engineering, QA, product, and design.

2. Product Vision

EventZen is a multi-portal event management platform that enables:

- Customers to discover events, register, pay, manage tickets, and request vendor access.
- Vendors and organizers to manage event operations, venues, check-in, finance, and reports.
- Admins to govern users, review access requests, manage platform operations, and control budgets, vendors, venues, and compliance actions.

3. Current Platform Scope

3.1 User Portals

The product now operates through three primary portals:

Portal	Primary Users	Purpose
Customer Portal	Attendees, customers	Discovery, checkout, tickets, registrations, account self-service, vendor access request

Portal	Primary Users	Purpose
Vendor Portal	Vendors, organizers	Event operations, venue booking, check-in, finance, reports
Admin Portal	Platform admins	User governance, request approval, events, venues, vendors, finance, reports

3.2 Role Model

The frontend normalizes backend roles into portal access:

- ADMIN -> Admin Portal
- VENDOR or ORGANIZER -> Vendor Portal
- ATTENDEE or CUSTOMER -> Customer Portal

Primary portal resolution currently prioritizes:

1. Admin
2. Vendor
3. Customer

4. Architecture Overview

4.1 Frontend

- React SPA
- TypeScript
- Vite
- Tailwind CSS
- React Router with guarded portal routes
- Lazy-loaded portal pages

4.2 Backend Microservices

Service	Technology	Primary Responsibility
Auth Service	Spring Boot	Authentication, profiles, roles, account requests, account lifecycle
Event Service	Spring Boot	Event catalog, event details, sessions, agenda, categories, status changes
Venue-Vendor Service	Node.js + Express	Venue inventory, booking availability, bookings,

Service	Technology	Primary Responsibility
Ticketing Service	ASP.NET Core	vendors, contracts Ticket types, registrations, tickets, waitlist, check-in, attendee import
Finance Service	Spring Boot	Budgets, budget items, approvals, expenses, payments, financial reports
Notification Service	Node.js + Express	Notifications, read state, delivery logs, preferences, templates, push tokens, webhooks

4.3 Data Stores

- MySQL for auth, event, and finance services
- MongoDB for venue-vendor, ticketing, and notification services

5. Frontend Product Definition

5.1 Public Routes

Route	Access	Purpose
/	Public	Marketing landing page and event discovery entry
/auth	Public	Login, registration, social sign-in, recovery entry
/events	Public	Event listing and filtering
/events/:id	Public	Event details, agenda, ticket selection

5.2 Authenticated Shared Routes

Route	Access	Purpose
/events/:id/checkout/:ticketTypeId	Authenticated	Checkout and registration flow
/my/tickets	Authenticated	Ticket wallet
/my/tickets/:registrationId/pass	Authenticated	Ticket pass view
/my/registrations	Authenticated	Customer registration history
/account/notifications	Authenticated	Notification inbox
/account/settings	Authenticated	Profile and account settings

5.3 Customer Portal

Route	Access	Purpose
/customer/dashboard	Customer Portal	Customer workspace dashboard

Implemented customer portal capabilities include:

- Registration overview and active registration count
- Spending summary using payment invoice data
- Check-in completion status
- Security checklist for email verification and MFA
- Self-service vendor access request submission
- Visibility into prior vendor-access requests and statuses

5.4 Vendor Portal

Route	Access	Purpose
/vendor/dashboard	Vendor Portal	Vendor summary dashboard
/vendor/events	Vendor Portal	Event operations workspace
/vendor/venues	Vendor Portal	Venue and booking workspace
/vendor/check-in	Vendor Portal	Live event check-in operations
/vendor/finance	Vendor Portal	Budget and finance workspace
/vendor/reports	Vendor Portal	Operational and financial reporting

Implemented vendor portal capabilities include:

- Event portfolio visibility
- Venue booking visibility
- Planned budget summary
- Quick access to event, venue, check-in, finance, and reporting workflows

5.5 Admin Portal

Route	Access	Purpose
/admin/dashboard	Admin Portal	Executive and governance dashboard
/admin/events	Admin Portal	Event administration
/admin/check-in	Admin Portal	Central check-in command

Route	Access	Purpose
		center
/admin/venues	Admin Portal	Venue administration
/admin/finance	Admin Portal	Finance oversight
/admin/reports	Admin Portal	Analytics and reporting
/admin/vendors	Admin Portal	Vendor administration

Implemented admin portal capabilities include:

- User management and role assignment by portal
- Vendor access request approval and rejection
- System account request review
- Direct account actions: deactivate, reactivate, GDPR delete
- Aggregated event and ticket visibility
- Administrative entry points into events, venues, check-in, finance, reports, and vendors

6. Current Functional Modules

6.1 Authentication and Identity

Current auth scope includes:

- Email/password registration
- Email/password login
- Google sign-in
- Refresh token flow
- Logout
- MFA setup and verification
- Forgot password and reset password
- Email verification resend and confirmation
- Authenticated current-user retrieval
- Authenticated profile update

6.2 Account Request Management

The platform now includes account request workflows not covered in the older PRD:

- User-submitted account requests
- My account request history
- Public reactivation request submission
- Public reactivation status lookup

- Admin review queue for account requests
- Admin approval and rejection actions
- Vendor access request flow from customer portal

Supported request types in the frontend and auth service:

- DEACTIVATE
- REACTIVATE
- GDPR_DELETE
- VENDOR_ACCESS

6.3 Event Management

Current event management scope includes:

- Event creation
- Event listing with filters
- Event detail retrieval
- Event update
- Event status transition
- Event archival
- Session creation
- Agenda retrieval
- Agenda reorder
- Search
- Category listing

6.4 Venue and Vendor Management

Current venue-vendor scope includes:

- Venue creation
- Venue listing
- Venue detail retrieval
- Venue update
- Venue deactivation
- Venue availability check with hall-level filtering
- Venue booking creation
- Venue bookings listing
- Vendor listing
- Vendor creation
- Vendor reviews

- Vendor hiring for events
- Contract status updates

6.5 Ticketing and Attendance

Current ticketing scope includes:

- Ticket type listing
- Ticket type creation
- Registration creation with idempotency
- My registrations
- Event registrations for admins/organizers
- Registration cancellation
- Ticket retrieval by id
- My ticket listing
- Waitlist join
- Check-in scan
- Check-in stats
- CSV attendee import

6.6 Finance and Budgeting

Current finance scope includes:

- Budget creation per event
- Budget retrieval
- Budget approval
- Budget item creation
- Expense logging
- Payment initiation
- Payment webhook handling
- Payment verification
- Financial report retrieval per event

6.7 Notification and Communication

The notification service is now both event-driven and API-driven. Implemented features include:

- Internal notification dispatch endpoint
- Notification inbox listing
- Notification detail retrieval
- Mark read/unread

- Soft delete of notifications
- Delivery log retrieval for admins
- Notification preferences
- Push token registration and removal
- Notification templates
- Template preview
- Admin webhook subscription management

7. Backend API Inventory

7.1 Auth Service APIs

Auth APIs

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register a user
POST	/api/v1/auth/login	Email/password login
POST	/api/v1/auth/google/login	Google sign-in
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/auth/logout	Logout with refresh token
GET	/api/v1/auth/me	Get current user
POST	/api/v1/auth/mfa/setup	Start MFA setup
POST	/api/v1/auth/mfa/verify	Verify MFA code
POST	/api/v1/auth/forgot-password	Request password reset
POST	/api/v1/auth/reset-password	Reset password
POST	/api/v1/auth/email-verification/resend	Resend verification
POST	/api/v1/auth/email-verification/confirm	Confirm email verification
PATCH	/api/v1/auth/me/profile	Update current profile

User Management APIs

Method	Endpoint	Purpose
GET	/api/v1/users	List users
PUT	/api/v1/users/{id}/roles	Assign roles
DELETE	/api/v1/users/{id}	Deactivate user
PATCH	/api/v1/users/{id}/reactivate	Reactivate user
DELETE	/api/v1/users/{id}/gdpr/delete	GDPR delete user

Account Request APIs

Method	Endpoint	Purpose
POST	/api/v1/account-requests	Submit account request
GET	/api/v1/account-requests/me	List my requests
POST	/api/v1/account-requests/public/reactivation	Public reactivation request
GET	/api/v1/account-requests/public/reactivation/status	Public reactivation status
DELETE	/api/v1/account-requests/{id}	Cancel my request
GET	/api/v1/account-requests/admin	Admin request queue
PATCH	/api/v1/account-requests/admin/{id}/approve	Approve request
PATCH	/api/v1/account-requests/admin/{id}/reject	Reject request

7.2 Event Service APIs

Method	Endpoint	Purpose
POST	/api/v1/events	Create event
GET	/api/v1/events	List events with filters
GET	/api/v1/events/{id}	Get event details
PUT	/api/v1/events/{id}	Update event
PATCH	/api/v1/events/{id}/status	Transition event status
DELETE	/api/v1/events/{id}	Archive event
POST	/api/v1/events/{id}/sessions	Add session
GET	/api/v1/events/{id}/agenda	Get agenda
PUT	/api/v1/events/{id}/agenda/reorder	Reorder agenda
GET	/api/v1/events/search	Search events
GET	/api/v1/categories	List categories

Supported list filters currently include:

- q
- category
- status

- city
- organizerId
- page
- size
- from
- to

7.3 Venue-Vendor Service APIs

Venue APIs

Method	Endpoint	Purpose
POST	/api/v1/venues	Create venue
GET	/api/v1/venues	List venues
GET	/api/v1/venues/bookings	List venue bookings
GET	/api/v1/venues/{id}	Get venue details
PUT	/api/v1/venues/{id}	Update venue
DELETE	/api/v1/venues/{id}	Deactivate venue
GET	/api/v1/venues/{id}/availability	Check availability
POST	/api/v1/venues/{id}/book	Create booking

Vendor and Contract APIs

Method	Endpoint	Purpose
GET	/api/v1/vendors	List vendors
POST	/api/v1/vendors	Create vendor
POST	/api/v1/vendors/{id}/reviews	Add vendor review
POST	/api/v1/events/{id}/vendors	Hire vendor for event
PATCH	/api/v1/contracts/{id}/status	Update contract status

Current venue service behavior also includes:

- Hall-aware availability logic
- Booking conflict detection
- Booking confirmation notification trigger

7.4 Ticketing Service APIs

Method	Endpoint	Purpose
GET	/api/v1/ticket-types?eventId={eventId}	List ticket types
POST	/api/v1/events/{eventId}/ticket-types	Create ticket type
POST	/api/v1/registrations	Register attendee

Method	Endpoint	Purpose
GET	/api/v1/registrations/me	My registrations
GET	/api/v1/events/{eventId}/registrations	Event registrations
DELETE	/api/v1/registrations/{registrationId}	Cancel registration
GET	/api/v1/tickets/{ticketId}	Get ticket
GET	/api/v1/tickets/me	List my tickets
POST	/api/v1/checkin/scan	Scan ticket for entry
GET	/api/v1/events/{eventId}/checkin/stats	Check-in stats
POST	/api/v1/events/{eventId}/waitlist	Join waitlist
POST	/api/v1/attendees/import	Import attendees via CSV
GET	/api/v1/health	Service health

Important implemented behaviors:

- Registration requires Idempotency-Key
- Ticket output includes signed QR payloads
- Waitlist auto-promotion is supported on cancellation

7.5 Finance Service APIs

Method	Endpoint	Purpose
POST	/api/v1/events/{eventId}/budget	Create event budget
GET	/api/v1/events/{eventId}/budget	Get event budget
PUT	/api/v1/budgets/{budgetId}/approve	Approve budget
POST	/api/v1/budgets/{budgetId}/items	Add budget item
POST	/api/v1/expenses	Log expense
POST	/api/v1/payments	Initiate payment
POST	/api/v1/payments/webhook	Payment webhook
POST	/api/v1/payments/verify	Verify payment
GET	/api/v1/events/{eventId}/reports/financial	Financial report

7.6 Notification Service APIs

Method	Endpoint	Purpose
POST	/notifications/send	Internal send endpoint
GET	/notifications	List notifications
GET	/notifications/delivery-logs	Delivery logs for admins
GET	/notifications/{id}	Notification details
PATCH	/notifications/{id}/read	Mark read/unread
DELETE	/notifications/{id}	Soft delete notification
GET	/notifications/preferences	Get preferences
POST	/notifications/preferences	Update preferences
GET	/notifications/push-tokens	List push tokens
POST	/notifications/push-tokens	Register push token
DELETE	/notifications/push-tokens	Remove push token
GET	/notifications/templates	List templates
POST	/notifications/templates	Create template
PUT	/notifications/templates/{id}	Update template
POST	/notifications/templates/{id}/preview	Preview template
GET	/notifications/webhooks	List webhook subscriptions
POST	/notifications/webhooks	Create webhook subscription
DELETE	/notifications/webhooks/{id}	Delete webhook subscription
GET	/health	Service health

8. Current User Journeys

8.1 Customer Journey

1. Browse events from landing page or event listing.
2. Open event details and inspect agenda and ticket options.
3. Log in or register if required.
4. Complete checkout for a selected ticket type.
5. View registrations and ticket wallet.

6. Use ticket pass and QR artifacts for event entry.
7. Manage notifications and account settings.
8. Optionally request vendor access from the customer portal.

8.2 Vendor Journey

1. Gain vendor or organizer access through admin role assignment or request approval.
2. Open vendor dashboard.
3. Create and manage events.
4. Browse venues, check availability, and create bookings.
5. Hire vendors and track contracts where applicable.
6. Configure ticketing and monitor registrations.
7. Run live check-in.
8. Monitor budgets, expenses, and reports.

8.3 Admin Journey

1. Open admin dashboard.
2. Review pending vendor and system account requests.
3. Assign or adjust user portal roles.
4. Perform direct lifecycle actions such as deactivate, reactivate, or GDPR delete.
5. Access event, venue, finance, vendor, and check-in admin surfaces.
6. Track aggregate operational activity and ticket volume.

9. Updated Requirements by Portal

9.1 Customer Portal Requirements

- Customers must be able to access a dedicated dashboard after login.
- Customers must be able to view active registrations and ticket wallet information.
- Customers must be able to review account security state such as email verification and MFA.
- Customers must be able to submit vendor access requests.
- Customers must be able to review notification inbox and account settings.

9.2 Vendor Portal Requirements

- Vendors must have a dedicated operational dashboard.
- Vendors must be able to access event, venue, check-in, finance, and reporting pages.
- Vendors must be able to see event portfolio and planned budget summaries.
- Vendors must be able to retrieve venue booking data limited to their operational scope.

9.3 Admin Portal Requirements

- Admins must be able to review and decide vendor access requests.
- Admins must be able to review deactivate, reactivate, and GDPR delete requests.
- Admins must be able to assign platform users into admin, vendor, or customer portals.
- Admins must be able to perform direct account actions.
- Admins must be able to access all operational modules.

10. Non-Functional Expectations

- Portal routes must enforce role-based access at the frontend.
- Backend services must enforce authenticated and role-based access for protected APIs.
- Ticket registration must remain idempotent.
- Check-in must support live scanning workflows.
- Notification APIs must support per-user read state and preference management.
- Finance operations must support payment verification and event-level reporting.
- Venue booking must support hall-aware availability checking.

11. Release Notes from Prior PRD Version

The following major changes have been incorporated since the prior PRD baseline:

- Product expanded from a primarily customer/admin framing to three formal portals: customer, vendor, and admin.
- Auth service expanded with Google login, forgot/reset password, email verification, profile update, and account-request workflows.
- Admin-facing governance workflows now include role reassignment, reactivation, deactivation, and GDPR deletion.
- Customer-facing vendor onboarding request flow is now implemented.
- Notification service now exposes user-facing and admin-facing APIs instead of being only a background consumer.
- Finance service now includes payment verification in addition to budgeting and expense tracking.
- Ticketing service now includes ticket types, my tickets, waitlist management, attendee import, and check-in statistics.
- Venue-vendor service now includes bookings list, venue detail/update/deactivate, reviews, and event-vendor hiring.

12. Source of Truth

This PRD update was derived from the current implementation in:

- Frontend route definitions and page modules
- Frontend API client modules
- Spring Boot controllers in auth, event, and finance services
- Express route modules in venue-vendor and notification services
- ASP.NET Core controllers and README in the ticketing service

13. Appendix

13.1 Key Frontend Route Groups

- Public: home, auth, event discovery, event details
- Shared authenticated: checkout, tickets, registrations, notifications, settings
- Customer portal: customer dashboard
- Vendor portal: dashboard, events, venues, check-in, finance, reports
- Admin portal: dashboard, events, check-in, venues, finance, reports, vendors

13.2 Recommended Next Documentation Follow-Ups

- Update system design document to reflect notification APIs and account-request workflows
- Update ERD to include account request, notification preference, push token, and webhook subscription entities
- Add explicit API gateway mapping documentation for all microservices
- Add sequence diagrams for customer checkout, vendor onboarding, and admin request approval